Human Verification: `Containers.Inventory`

Inventory as an Update Monad

Machine-generated from `lean/Inventory.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.Inventory` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

The file verifies §8 of the paper: directed containers with *constant position sets* are exactly *monoid actions*, and these co-interpret as *update monads* $TX = S \to M \times X$. Kleisli maps observe a stock level in S and emit a movement in M. Two paper remarks are machine-checked: abelianisation forgets audit-trail order, and monus clamps on $\mathbb{N}$ while $\mathbb{Z}$ admits backorders.

Contents

1 Overview **2**

2 Monoid actions **2**
2.1 Structure: `MonoidAction` . . . 2

3 Constant-position directed containers **2**
3.1 Structure: `ConstDC` . . . 2
3.2 Definitions: `ConstDC.toAction` and `MonoidAction.toConst` . . . 3
3.3 Theorems: `MonoidAction.toConst_toAction` and `ConstDC.toAction_toConst` 4
3.4 Theorem: `transport_const` . . . 4
3.5 Definitions: `ConstDC.toDirected` and `MonoidAction.toDC` . . . 4

4 The update monad **5**
4.1 Definition: `Update` . . . 5
4.2 Definitions: `Update.pure` and `Update.bind` . . . 5
4.3 Theorems: monad laws . . . 6

5 Paper remarks, machine-checked **6**
5.1 Example: abelianisation forgets audit-trail order . . . 6
5.2 Theorems: monus clamps; $\mathbb{Z}$ admits backorders . . . 7

6 Concrete instance: two-product warehouse **7**
6.1 Definitions: `intPairMonoid` and `inventory` . . . 7
6.2 Definitions: `warehouseDC`, `receive`, and `policy` . . . 8
6.3 Sanity checks (examples checked by `decide`) . . . 8

7 Translation conventions **9**

1 Overview

In a directed container $\mathcal{D}$ where $\mathrm{Pos}(s) = M$ for all shapes s (positions independent of shape), the shift $\mathbf{shift}(s, p, q)$ is genuinely a binary operation $M \times M \to M$ and does not depend on s. The five directed-container laws then collapse exactly to the axioms of a monoid action: M is the acting monoid, $S = \mathrm{Shape}$ is the set of states.

The update monad $T_A X = A.S \to A.\mathrm{Mon}.M \times X$ is the co-interpretation of this directed container: a Kleisli map looks at the current state, emits a movement, and returns a value.

2 Monoid actions

2.1 Structure: `MonoidAction`

Definition 2.1 (Bundled right monoid action)**.** A **bundled right monoid action** A consists of:

- a monoid $A.\texttt{Mon}$ of *stock movements*;
- a type $A.S$ of *states* (stock levels);
- an action $A.\texttt{act} : A.S \to A.\texttt{Mon}.M \to A.S$;
- a proof of `act_one`: $A.\texttt{act}(s, 1) = s$; and
- a proof of `act_mul`: $A.\texttt{act}(A.\texttt{act}(s, m), m') = A.\texttt{act}(s, m \cdot m')$.

Lean.

```
structure MonoidAction where
  Mon     : Monoid'
  S       : Type
  act     : S → Mon.M → S
  act_one : ∀ s, act s Mon.one = s
  act_mul : ∀ s m m', act (act s m) m' = act s (Mon.mul m m')
```

Haskell.

```
-- A right monoid action:
class Monoid m ⇒ Action s m where
  act :: s → m → s
  -- act_one: act s mempty = s
  -- act_mul: act (act s m) m' = act s (mappend m m')
```

3 Constant-position directed containers

3.1 Structure: `ConstDC`

Definition 3.1 (Constant-position directed container)**.** A **constant-position directed container** C has:

- a type $C.S$ of shapes (states);
- a single position type $C.M$ (independent of the shape);
- $C.\texttt{sub} : C.S \to C.M \to C.S$ (state transition);
- $C.\texttt{root} : C.M$ (unit movement); and
- $C.\texttt{shift} : C.M \to C.M \to C.M$ (movement composition).

Five laws (c1–c5) correspond to the directed-container laws D1–D5 with all shape-dependent transports removed: the D2/D5 casts vanish because position types are constant.

Lean.

```
structure ConstDC where
  S     : Type
  M     : Type
  sub   : S → M → S
  root  : M
  shift : M → M → M
  c1 : ∀ s, sub s root = s
  c2 : ∀ m, shift root m = m
  c3 : ∀ m, shift m root = m
  c4 : ∀ s p q, sub s (shift p q) = sub (sub s p) q
  c5 : ∀ p q r, shift (shift p q) r = shift p (shift q r)
```

Haskell.

```
data ConstDC s m = ConstDC
  { cdcSub   :: s → m → s
  , cdcRoot  :: m
  , cdcShift :: m → m → m
  -- c1..c5 as law comments
  }
```

Fidelity note. The key insight of this section is that with constant positions, the shift $C.\texttt{shift} : C.M \to C.M \to C.M$ is a genuine binary operation on a single set M — giving a monoid $(M, C.\texttt{shift}, C.\texttt{root})$ with laws c2/c3/c5. For varying $\mathrm{Pos}(s)$, the shift is only a fibrewise composition and no global monoid exists. This is the *sound* case of "the monoid that wasn't" (noted in the dream journal).

3.2 Definitions: `ConstDC.toAction` and `MonoidAction.toConst`

Definition 3.2 (Dictionary between `ConstDC` and `MonoidAction`)**.** The two structures are definitionally isomorphic:

- `ConstDC.toAction`(C): the monoid is $(C.M, C.\texttt{shift}, C.\texttt{root})$; the action is $C.\texttt{sub}$; laws c2/c3/c5 give monoid laws; laws c1/c4 give action laws.
- `MonoidAction.toConst`(A): $S = A.S$, $M = A.\texttt{Mon}.M$, $\texttt{sub} = A.\texttt{act}$, $\texttt{root} = A.\texttt{Mon.one}$, $\texttt{shift} = A.\texttt{Mon.mul}$.

Lean.

```
def ConstDC.toAction (C : ConstDC) : MonoidAction where
  Mon := { M := C.M, mul := C.shift, one := C.root,
           mul_assoc := C.c5, one_mul := C.c2, mul_one := C.c3 }
  S := C.S; act := C.sub
  act_one := C.c1
  act_mul := fun s m m' ⇒ (C.c4 s m m').symm

def MonoidAction.toConst (A : MonoidAction) : ConstDC where
  S := A.S; M := A.Mon.M
  sub := A.act; root := A.Mon.one; shift := A.Mon.mul
  c1 := A.act_one; c2 := A.Mon.one_mul; c3 := A.Mon.mul_one
  c4 := fun s p q ⇒ (A.act_mul s p q).symm
  c5 := A.Mon.mul_assoc
```

Haskell.

```
toAction :: ConstDC s m → (s, m, m → m → m)
toAction c = (cdcSub c, cdcRoot c, cdcShift c)
```

```
toConst :: Action s m ⇒ s → m → ConstDC s m
toConst _ _ = ConstDC act mempty mappend
```

3.3 Theorems: `MonoidAction.toConst_toAction` and `ConstDC.toAction_toConst`

Theorem 3.3 (Dictionary round trips)**.** The dictionaries are mutually inverse (definitionally):

1. A.`toConst.toAction` $= A$.
2. C.`toAction.toConst` $= C$.

Both hold by `rfl` (structure eta and proof irrelevance).

Lean.
```
theorem MonoidAction.toConst_toAction (A : MonoidAction) :
    A.toConst.toAction = A := rfl

theorem ConstDC.toAction_toConst (C : ConstDC) :
    C.toAction.toConst = C := rfl
```

Haskell.
```
-- toConst . toAction = id
-- toAction . toConst = id
```

3.4 Theorem: `transport_const`

Theorem 3.4 (Constant-fibre transport is identity)**.** If the motive of a transport is constant (does not depend on the shape), then the transport is the identity:

$$\forall\, h : s = t,\ q : M, \quad \mathrm{transport}_M(h, q) = q.$$

This is the key lemma explaining why D2/D5 casts vanish in the constant- position setting.

Lean.
```
theorem transport_const {S M : Type} :
    ∀ {s t : S} (h : s = t) (q : M),
    @Eq.rec S s (fun _ _ ⇒ M) q t h = q
```

Haskell.
```
-- In Haskell, transport along equalities does not exist
-- (no propositional equality at the term level).
-- This theorem is vacuously true: there is no Eq.rec to reason about.
```

3.5 Definitions: `ConstDC.toDirected` and `MonoidAction.toDC`

Definition 3.5 (Embedding into DirectedContainer)**.** `ConstDC.toDirected`(C) embeds a constant-position directed container into the general `DirectedContainer` type by setting $\mathrm{Pos}(s) := C.M$ for all shapes s. D1–D5 are exactly c1–c5 (with D2 and D5 augmented by `transport_const` to handle the casts).

`MonoidAction.toDC`(A) := A.`toConst.toDirected` is the corresponding `DirectedContainer` for a monoid action.

Lean.
```
def ConstDC.toDirected (C : ConstDC) : DirectedContainer where
  Shape := C.S; Pos := fun _ ⇒ C.M
  root := fun _ ⇒ C.root
  sub := C.sub
```

```
  shift := fun _ p q ⇒ C.shift p q
  d1 := C.c1; d4 := C.c4; d3 := fun _ p ⇒ C.c3 p
  d2 := fun s q ⇒
    (C.c2 q).trans (transport_const (C.c1 s) q).symm
  d5 := fun s p q q' ⇒
    (congrArg (C.shift (C.shift p q))
              (transport_const (C.c4 s p q).symm q')).trans
      (C.c5 p q q')

def MonoidAction.toDC (A : MonoidAction) : DirectedContainer :=
  A.toConst.toDirected
```

Haskell.

```
toDC :: Action s m ⇒ s → DirectedContainer s m
toDC _ = DirectedContainer
  { dcPos   = const mempty  -- position type is always M
  , dcRoot  = mempty
  , dcSub   = act
  , dcShift = \_ → mappend
  }
```

4 The update monad

4.1 Definition: `Update`

Definition 4.1 (Update monad)**.** The **update monad** of a monoid action A is:

$$T_A\,X \;=\; A.S \to A.\mathtt{Mon}.M \times X.$$

A value of type $T_A\,X$ is a *Kleisli map*: it observes a current state in $A.S$, emits a movement in $A.\mathtt{Mon}.M$, and returns a value in X. This is the co-interpretation of the constant-position directed container $A.\mathtt{toDC}$ (Ahman–Uustalu, *Update monads*).

Lean.

```
def Update (A : MonoidAction) (X : Type) : Type :=
  A.S → A.Mon.M * X
```

Haskell.

```
newtype Update s m x = Update { runUpdate :: s → (m, x) }
-- cf. Data.Functor.Product
```

4.2 Definitions: `Update.pure` and `Update.bind`

Definition 4.2 (Monad operations)**.**

- $\mathtt{pure}(A, x) : T_A\,X$ — emit no movement, return x: $\mathtt{pure}(A, x)(s) = (1, x)$.
- $\mathtt{bind}(t, k) : T_A\,Y$ for $t : T_A\,X$, $k : X \to T_A\,Y$ — run t, apply the emitted movement to the state, run k on the updated state, multiply the two movements:

$$\mathtt{bind}(t,k)(s) \;=\; \big(t(s).\mathbf{1} \cdot k(t(s).\mathbf{2})(A.\mathtt{act}(s,\, t(s).\mathbf{1})).\mathbf{1},\; k(t(s).\mathbf{2})(A.\mathtt{act}(s,\, t(s).\mathbf{1})).\mathbf{2}\big).$$

Lean.

```
def pure (A : MonoidAction) (x : X) : Update A X :=
  fun _ ⇒ (A.Mon.one, x)

def bind (t : Update A X) (k : X → Update A Y) : Update A Y :=
  fun s ⇒
    (A.Mon.mul (t s).1 (k (t s).2 (A.act s (t s).1)).1,
     (k (t s).2 (A.act s (t s).1)).2)
```

Haskell.

```haskell
instance (Monoid m, Action s m) ⇒ Monad (Update s m) where
  return x = Update $ \_ → (mempty, x)
  Update t >>= k = Update $ \s →
    let (m,  x) = t s
        (m', y) = runUpdate (k x) (act s m)
    in  (mappend m m', y)
```

4.3 Theorems: monad laws

Theorem 4.3 (Monad laws for `Update`)**.** 1. **Left unit**: $\texttt{bind}(\texttt{pure}(A, x), k) = k(x)$. Uses `act_one` (the state is unchanged by the unit movement) and `one_mul`.

2. **Right unit**: $\texttt{bind}(t, \texttt{pure}(A)) = t$. Uses `mul_one`.

3. **Associativity**: $\texttt{bind}(\texttt{bind}(t, k), h) = \texttt{bind}(t, \lambda x.\, \texttt{bind}(k(x), h))$. Uses `act_mul` and `mul_assoc`.

Lean.

```lean
theorem pure_bind (x : X) (k : X → Update A Y) :
    bind (pure A x) k = k x

theorem bind_pure (t : Update A X) : bind t (pure A) = t

theorem bind_assoc (t : Update A X) (k : X → Update A Y)
    (h : Y → Update A Z) :
    bind (bind t k) h = bind t (fun x ⇒ bind (k x) h)
```

Haskell.

```haskell
-- Standard monad laws:
-- return x >>= k       = k x          (left unit)
-- t >>= return         = t             (right unit)
-- (t >>= k) >>= h      = t >>= (\x → k x >>= h)   (associativity)
```

5 Paper remarks, machine-checked

5.1 Example: abelianisation forgets audit-trail order

Proposition 5.1 (Abelianisation forgets order)**.** Two movement words with the same net balance can differ as audit trails, and can have different operational consequences. Specifically:

$$\mathrm{net}[\mathrm{receipt}(2), \mathrm{issue}(1)] \;=\; \mathrm{net}[\mathrm{issue}(1), \mathrm{receipt}(2)]$$

(both equal 1), yet $[\mathrm{receipt}(2), \mathrm{issue}(1)] \neq [\mathrm{issue}(1), \mathrm{receipt}(2)]$ as lists. The second word passes through a shortage (from zero stock) that the first never sees.

Lean.

```lean
example :
    net [.receipt 2, .issue 1] = net [.issue 1, .receipt 2] /\
    ([Movement.receipt 2, .issue 1] ≠ [.issue 1, .receipt 2]) := by
  decide
```

Haskell.

```haskell
-- net [Receipt 2, Issue 1] = net [Issue 1, Receipt 2]  -- both = 1
-- but [Receipt 2, Issue 1] /= [Issue 1, Receipt 2]     -- as lists
```

5.2 Theorems: monus clamps; $\mathbb{Z}$ admits backorders

Proposition 5.2 (Monus clamps on $\mathbb{N}$)**.** Truncated subtraction on $\mathbb{N}$ silently discards unmet demand: $\exists s, m : \mathbb{N},\ (s - m) + m \neq s$. Witness: $s = 0$, $m = 1$.

Theorem 5.3 ($\mathbb{Z}$ admits backorders)**.** For all $s, m : \mathbb{Z}$, $(s - m) + m = s$.

Lean.

```
example : exists s m : Nat, (s - m) + m ≠ s := ⟨0, 1, by decide⟩

theorem int_sub_add_cancel : ∀ s m : Int, (s - m) + m = s := by
  intro s m; omega
```

Haskell.

```
-- Nat monus: (0 - 1) + 1 = 0 /= 0+1 = 1  -- no, (0 - 1) = 0 in Nat
-- so  (0 - 1) + 1 = 1 /= 0   -- True: Nat monus clamps
monusClamps :: Bool
monusClamps = ((0 :: Word) - 1) + 1 /= 0  -- True

-- Int: (s - m) + m = s always
intCancel :: Integer → Integer → Bool
intCancel s m = (s - m) + m == s
```

6 Concrete instance: two-product warehouse

6.1 Definitions: `intPairMonoid` and `inventory`

Definition 6.1 (Integer-pair monoid and inventory action)**.** The **integer-pair monoid** is $(\mathbb{Z} \times \mathbb{Z}, +, (0, 0))$ (componentwise addition). The **inventory action** is the right action of $\mathbb{Z}^2$ on $\mathbb{Z}^2$ by componentwise addition: $\texttt{act}((s_1, s_2), (m_1, m_2)) = (s_1 + m_1, s_2 + m_2)$. Negative stock levels represent backorders (accepted-but-unfulfilled demand).

Lean.

```
@[reducible] def intPairMonoid : Monoid' where
  M := Int * Int
  mul a b := (a.1 + b.1, a.2 + b.2)
  one := (0, 0)
  ...

@[reducible] def inventory : MonoidAction where
  Mon := intPairMonoid
  S := Int * Int
  act s m := (s.1 + m.1, s.2 + m.2)
  ...
```

Haskell.

```
type Stock = (Integer, Integer)

instance Semigroup Stock where
  (a1, a2) <> (b1, b2) = (a1+b1, a2+b2)

instance Monoid Stock where
  mempty = (0, 0)

instance Action Stock Stock where
  act (s1, s2) (m1, m2) = (s1+m1, s2+m2)
```

6.2 Definitions: `warehouseDC`, `receive`, and `policy`

Definition 6.2 (Warehouse DC, replenishment policy)**.** The **warehouse directed container** `warehouseDC` := `inventory.toDC` is the directed container with stock levels as shapes and $\mathbb{Z}^2$-movements as positions.

`receive`$(m) : T_{\texttt{inventory}}\,()$ is a Kleisli map that unconditionally emits the movement m.

The **order-up-to replenishment policy** is a Kleisli map: for each product, if stock is below 5, order up to 10:

$$\texttt{policy}((), s) \;=\; \big((\text{if } s_1 < 5 \text{ then } 10 - s_1 \text{ else } 0,\ \text{if } s_2 < 5 \text{ then } 10 - s_2 \text{ else } 0),\ ()\big).$$

Lean.

```
def warehouseDC : DirectedContainer := inventory.toDC

@[reducible] def receive (m : Int * Int) : Update inventory Unit :=
  fun _ ⇒ (m, ())

@[reducible] def policy : Unit → Update inventory Unit :=
  fun _ s ⇒
    ((if s.1 < 5 then 10 - s.1 else 0,
      if s.2 < 5 then 10 - s.2 else 0), ())
```

Haskell.

```
type WarehouseM = Update Stock Stock

receive :: Stock → WarehouseM ()
receive m = Update $ \_ → (m, ())

policy :: () → WarehouseM ()
policy () = Update $ \(s1, s2) →
  ((if s1 < 5 then 10 - s1 else 0,
    if s2 < 5 then 10 - s2 else 0), ())
```

6.3 Sanity checks (examples checked by `decide`)

Proposition 6.3 (Policy with product 1 low, product 2 high)**.** With stocks $(2, 7)$ (product 1 below reorder, product 2 above):

- the policy emits $(8, 0)$ (order 8 of product 1, nothing of 2);
- applying the emitted order brings stocks to $(10, 7)$.

Proposition 6.4 (Kleisli composition threads state)**.** Receiving 3 units of product 1 before the policy, starting from stocks $(1, 7)$: the policy sees stock 4 $(= 1 + 3)$ and orders 6; total emitted movement is $(9, 0)$.

Lean.

```
example : (policy () (2, 7)).1 = (8, 0) := by decide
example : inventory.act (2, 7) (policy () (2, 7)).1 = (10, 7) := by
   decide
example :
    Update.bind (receive (3, 0)) (fun _ ⇒ policy ()) (1, 7)
      = ((9, 0), ()) := by decide
```

Haskell.

```
-- runUpdate (policy ()) (2, 7) = ((8, 0), ())
-- act (2,7) (8,0) = (10,7)
-- runUpdate (receive (3,0) >>= (\_ → policy ())) (1,7) = ((9,0),())
```

7 Translation conventions

1. **Bundled vs. typeclass.** Lean uses bundled records for `Monoid'` and `MonoidAction`; Haskell uses typeclasses. The mathematical content is the same.
2. **Transport in D2/D5.** In the general directed-container formalism, D2 and D5 involve transports along shape equalities. In the constant-position case, these transports are the identity (`transport_const`), so c2 and c5 are the bare monoid laws without casts.
3. **Backorders as negative stock.** The paper notes that $\mathbb{N}$ monus clamps while $\mathbb{Z}$ is honest. The Lean formalisation uses $\mathbb{Z}$ throughout the warehouse instance.
4. **Proof bodies omitted.** Only definitions and theorem *statements* are translated. Lean checks the proofs; the human verifier checks that the *statements* match the intended mathematics.